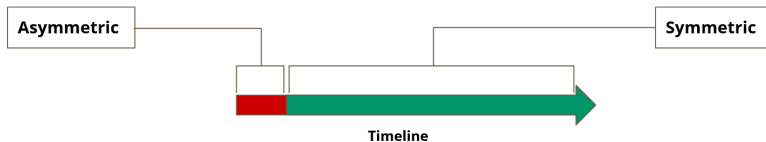


## Session 5: Diffie-Hellman

Slides prepared by UMBC Protocol Analysis Lab

Last edited June 26, 2026

# How do we actually use encryption?



# Generators

# Generators

Let  $p$  be a prime, and  $g \in \{1, \dots, p-1\}$ .

# Generators

Let  $p$  be a prime, and  $g \in \{1, \dots, p-1\}$ .

## Definition

$g$  is a *generator* for  $\text{mod } p$  if:

$$(\{g^1, \dots, g^{(p-1)}\} \text{ mod } p) = \{1, \dots, p-1\}$$

This set  $\{1, \dots, p-1\}$  on multiplication is known as  $\mathbb{Z}_p^\times$

# Generators

Let  $p$  be a prime, and  $g \in \{1, \dots, p-1\}$ .

## Definition

$g$  is a *generator* for  $\text{mod } p$  if:

$$(\{g^1, \dots, g^{(p-1)}\} \text{ mod } p) = \{1, \dots, p-1\}$$

This set  $\{1, \dots, p-1\}$  on multiplication is known as  $\mathbb{Z}_p^\times$

We can represent a generator in CPSA as `(gen)`,  
or `(exp (gen) (one))`.

# Generators

Let  $p$  be a prime, and  $g \in \{1, \dots, p-1\}$ .

## Definition

$g$  is a *generator* for  $\text{mod } p$  if:

$$(\{g^1, \dots, g^{(p-1)}\} \text{ mod } p) = \{1, \dots, p-1\}$$

This set  $\{1, \dots, p-1\}$  on multiplication is known as  $\mathbb{Z}_p^\times$

We can represent a generator in CPSA as `(gen)`,

or `(exp (gen) (one))`.

Note that `(gen)` is a term itself in CPSA.

# Exponents



# Exponents

Recall the exponent property

$$x^a \cdot x^b = x^{a+b}$$

# Exponents

Recall the exponent property

$$x^a \cdot x^b = x^{a+b}$$

Diffie-Hellman takes advantage of this property by letting us intelligently choose  $a$  and  $b$ .

# Exponents

Recall the exponent property

$$x^a \cdot x^b = x^{a+b}$$

Diffie-Hellman takes advantage of this property by letting us intelligently choose  $a$  and  $b$ . We call these exponent terms  $a$  and  $b$  random exponents, with *sort* rndx.

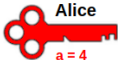
## Public Channel

1. Alice and Bob agree on public parameters

$$p = 23, g = 5$$



Alice



$$a = 4$$

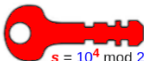


2. Alice combines her secret key (a) with the parameters and sends the resulting public key (A) to Bob



$$A = 5^4 \bmod 23 = 4$$

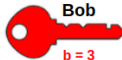
4. Alice combines (B) with her secret key (a)



$$s = 10^4 \bmod 23 = 18$$



Bob



$$b = 3$$



3. Bob combines his secret key (b) with the parameters and sends the resulting public key (B) to Alice



$$B = 5^3 \bmod 23 = 10$$

5. Bob combines (A) with his secret key (b)



$$s = 4^3 \bmod 23 = 18$$

6. Alice and Bob have a shared secret!

# Diffie-Hellman

Alice

Bob

● = Reception      ● = Transmission

# Diffie-Hellman

Alice

Let Alice have

$$a \in \{1, \dots, p-1\}$$

and  $g, p$

$$\text{Let } A = g^a \bmod p$$

Bob

Let Bob have

$$b \in \{1, \dots, p-1\}$$

and  $g, p$

$$\text{Let } B = g^b \bmod p$$

● = Reception

● = Transmission

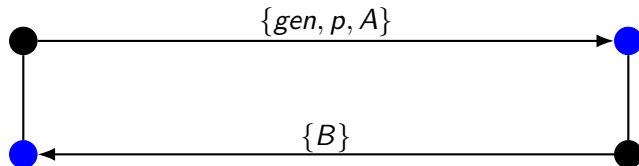
# Diffie-Hellman

Alice

Bob

Let Alice have  
 $a \in \{1, \dots, p-1\}$   
and  $g, p$   
Let  $A = g^a \bmod p$

Let Bob have  
 $b \in \{1, \dots, p-1\}$   
and  $g, p$   
Let  $B = g^b \bmod p$



= Reception



= Transmission

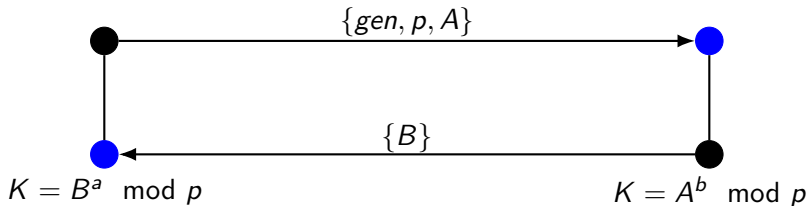
# Diffie-Hellman

Alice

Bob

Let Alice have  
 $a \in \{1, \dots, p-1\}$   
and  $g, p$   
Let  $A = g^a \bmod p$

Let Bob have  
 $b \in \{1, \dots, p-1\}$   
and  $g, p$   
Let  $B = g^b \bmod p$



$$\begin{aligned} K &= A^b \bmod p = (g^a \bmod p)^b \\ &= g^{ab} \bmod p \\ &= (g^b \bmod p)^a = B^a \bmod p \end{aligned}$$

● = Reception      ● = Transmission



# Diffie-Hellman Assumptions

# Diffie-Hellman Assumptions

States that its computationally infeasible to compute  $g^{(ab)}$  even if  $g, g^a, g^b$  are known.

# Diffie-Hellman Assumptions

States that its computationally infeasible to compute  $g^{(ab)}$  even if  $g, g^a, g^b$  are known.

This is how it's possible to exchange keys using Diffie - Hellman securely.

# Diffie-Hellman Assumptions

States that its computationally infeasible to compute  $g^{(ab)}$  even if  $g, g^a, g^b$  are known.

This is how it's possible to exchange keys using Diffie - Hellman securely.

There is a catch:

# Diffie-Hellman Assumptions

States that its computationally infeasible to compute  $g^{(ab)}$  even if  $g, g^a, g^b$  are known.

This is how it's possible to exchange keys using Diffie - Hellman securely.

There is a catch: whether it is actually computationally infeasible is not known, it is an open problem.

## Some CPSA hints and notes

- ▶ Need correct origination assumptions
  - ▶ 'a' and 'b' (of sort `rndx`) need to be `pen-non-orig`
  - ▶ `msg` should be `uniq-orig` (if not it could lead to many shapes)
- ▶ CPSA is used to find structural flaws, not algebraic flaws

## Some CPSA hints and notes

- ▶ Need correct origination assumptions
  - ▶ 'a' and 'b' (of sort `rndx`) need to be `pen-non-orig`
  - ▶ `msg` should be `uniq-orig` (if not it could lead to many shapes)
- ▶ CPSA is used to find structural flaws, not algebraic flaws
- ▶ Now let's take a look at the model!

# Reading Protocol Specifications

What do the keywords mean?



# RFC 2119

- ▶ Scott Bradner (March 1997) [Key words for use in RFCs to Indicate Requirement Levels](#)

# RFC 2119

- ▶ Scott Bradner (March 1997) [Key words for use in RFCs to Indicate Requirement Levels](#)
- ▶ **MUST** indicates that a portion of the spec is absolutely required.

# RFC 2119

- ▶ Scott Bradner (March 1997) [Key words for use in RFCs to Indicate Requirement Levels](#)
- ▶ **MUST** indicates that a portion of the spec is absolutely required.
- ▶ **MUST NOT** is an absolute prohibition of something.

# RFC 2119

- ▶ Scott Bradner (March 1997) [Key words for use in RFCs to Indicate Requirement Levels](#)
- ▶ **MUST** indicates that a portion of the spec is absolutely required.
- ▶ **MUST NOT** is an absolute prohibition of something.
- ▶ **SHOULD** indicates that something is recommended.

# RFC 2119

- ▶ Scott Bradner (March 1997) [Key words for use in RFCs to Indicate Requirement Levels](#)
- ▶ **MUST** indicates that a portion of the spec is absolutely required.
- ▶ **MUST NOT** is an absolute prohibition of something.
- ▶ **SHOULD** indicates that something is recommended.
- ▶ **SHOULD NOT** indicates that something is not recommended.

# RFC 2119

- ▶ Scott Bradner (March 1997) [Key words for use in RFCs to Indicate Requirement Levels](#)
- ▶ **MUST** indicates that a portion of the spec is absolutely required.
- ▶ **MUST NOT** is an absolute prohibition of something.
- ▶ **SHOULD** indicates that something is recommended.
- ▶ **SHOULD NOT** indicates that something is not recommended.
- ▶ **MAY** indicates that something is optional.

# What really happens

- ▶ **MUST** actually gets included

# What really happens

- ▶ **MUST** actually gets included
- ▶ Nothing else is included ...



# Some tips for reading protocol specifications

- ▶ FIND THE SECURITY GOALS
- ▶ Find the messages
  - ▶ “The [party1] relays [data] to [party2]”
  - ▶ “[party2] receives [data]” (figure out who sent it)
  - ▶ “With [data], [party2] does [action]” (how did it get data?)
- ▶ Find what every implementation must consider (*MUST*), especially if you are comparing.
- ▶ Find assumptions in the protocol (DY adversary)
  - ▶ “With private keys kept secret. . .”
  - ▶ “We assume both clients are honest participants”
  - ▶ “A high entropy source is available” (UUID4)

# Homework

# Capture the Flag Exercises

Use CPSA to model protocols that are written in Python, identify the attack, and write the code needed to “capture the flag” (get the secret value out of the exchange).

# Survey

Fill in details about the survey here!